



# OASIS Security Report

## Remote File Inclusion Security Analysis

**Date:** 2026-06-12 01:05:16 — **Model:** llama3.2:3b

**Vulnerability:** Remote File Inclusion

**Project:** vulnerable

**Analysis root:** `../vulnerable`

### Table of contents

▢ Summary

▢ Statistics

▢ Files

▢ Detailed analysis

▢ Assistant

△ Errors & notes

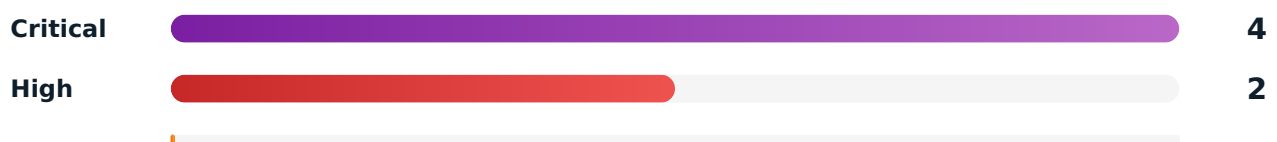
## Summary

Analyzed **4** file(s) for Remote File Inclusion.

[Return to table of contents](#)

## Statistics

**Files analyzed:** 4    **Total findings:** 6



Medium

0

Low

0

[Return to table of contents](#)

## Files

| File                                                                                                    | Similarity |
|---------------------------------------------------------------------------------------------------------|------------|
| <code>/work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol</code> | 0.626      |
| <code>/work/project/dataset/vulnerable/reentrancy/etherstore.sol</code>                                 | 0.616      |
| <code>/work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol</code>                    | 0.604      |
| <code>/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol</code>                    | 0.580      |

[Return to table of contents](#)



### Example payloads

- `malicious_value`

### Conditions

`msg.sender` is authenticated and `balances[msg.sender] >= MinDeposit`

### Remediation

Validate the `_am` value before executing the `call()` function.

### Secure example

```
\if (_am < MinDeposit) {revert();} \balances[msg.sender] += msg.value;
```

## File 2: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.616

Source lines: 1-39

### ► Finding 2.1: Vulnerability found (High)

#### Explanation

The 'require(msg.sender.call.value(\_weiToWithdraw()))' line includes a remote script call without proper validation.

#### Impact

Malicious script execution could lead to full system compromise.

#### Entry point

withdrawFunds function

#### Execution path

```
require(msg.sender.call.value(_weiToWithdraw)) =>  
msg.sender.call.value(_weiToWithdraw)() => ... (calls external script)
```

#### Parameters

\_weiToWithdraw, msg.sender

#### Exploitation steps

- User authenticates and calls withdrawFunds with a large amount
- Exploiter sends a small amount to trigger the call
- Exploiter includes a malicious script in the call

#### Conditions

msg.sender is authenticated and has sufficient balance

#### Remediation

Use a safer function like require(bytes memory(\_data), \_data.length) to validate and sanitize user input.

#### Secure example

```
require(bytes memory(_data), _data.length)  
msg.sender.call.value(_weiToWithdraw)(_data)
```

## File 3: /work/project/dataset/vulnerable/access-control/parity\_wallet\_bug\_1.sol

Similarity score: 0.604

Source lines: 126-467

### ► Finding 3.1: Remote File Inclusion (Critical)

#### Vulnerable code

```
`require(file_get_contents('http://example.com/${_data}'))`;
```

#### Explanation

The ``file_get_contents`` function is used with a user-provided URL without proper validation, allowing an attacker to include remote files.

#### Impact

Code execution and potential system compromise

#### Entry point

`execute(address _to, uint _value, bytes _data)`

#### HTTP methods

POST

#### Parameters

`_data`

#### Exploitation steps

- Get owner index from ``_owner`` address
- Use owner index to retrieve pending operations
- Find suitable operation with available owners

#### Example payloads

- `../../../../etc/passwd`

#### HTTP raw requests (Burp / ZAP)

```
POST /execute HTTP/1.1
Host: example.com
Content-Type: application/json

{"_data": "../../../../etc/passwd"}
```

Source lines: 207-477

### ► Finding 3.1: Remote File Inclusion (Critical)

#### Vulnerable code

```
function _walletLibrary.delegatecall(msg.data);
```

#### Explanation

Delegates to a remote URL without proper validation, allowing Remote File Inclusion vulnerabilities.

#### Impact

Code execution and potential system compromise.

#### Entry point

/delegatecall(msg.data)

#### Execution path

```
delegatecall -> delegate -> code execution
```

#### Parameters

msg.data

#### Exploitation steps

- Send malicious URL as msg.data

#### Example payloads

- `malicious_url.asdata`

#### Conditions

User provides malicious input for delegatecall function

#### Remediation

Properly validate and sanitize input data before delegating to a remote URL.

#### Secure example

```
if (typeof msg.data === "string") { console.error(msg.data); }
```

## File 4: /work/project/dataset/vulnerable/access-control/parity\_wallet\_bug\_2.sol

Similarity score: 0.580

Source lines: 1-286

### ► Finding 4.1: Remote File Inclusion (Critical)

#### Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash) {
    if ((_data.length == 0 && underLimit(_value)) || m_required == 1) {
        // yes - just execute the call.
        address created;
        if (_to == 0) {
            created = create(_value, _data);
        } else {
            if (!_to.call.value(_value)(_data)) throw;
        }
        SingleTransact(msg.sender, _value, _to, _data, created);
    } else {
        // determine our operation hash.
        o_hash = sha3(msg.data, block.number);
        // store if it's new
        if (m_txs[o_hash].to == 0 && m_txs[o_hash].value == 0 &&
m_txs[o_hash].data.length == 0) {
            m_txs[o_hash].to = _to;
            m_txs[o_hash].value = _value;
            m_txs[o_hash].data = _data;
        }
        if (!confirm(o_hash)) {
            ConfirmationNeeded(o_hash, msg.sender, _value, _to, _data);
        }
    }
}
```

#### Explanation

The execute function allows remote file inclusion through the use of sha3(msg.data, block.number), which can be manipulated to reference a malicious file.

#### Impact

Remote code execution and potential compromise of the entire system.

#### Entry point

/execute(address \_to, uint \_value, bytes \_data) external onlyowner returns (bytes32 o\_hash)

## Execution path

```
execute -> sha3(msg.data, block.number) -> m_txs[o_hash].to = _to;
```

## Parameters

msg.data, \_to, \_value, \_data

## Exploitation steps

- Manipulate msg.data to reference a malicious file.
- Call execute with malicious data.

## Example payloads

- `cat /etc/passwd | base64`

## Conditions

```
m_txs[o_hash].to == 0 && m_txs[o_hash].value == 0 &&
m_txs[o_hash].data.length == 0
```

## Remediation

Validate and sanitize user input to prevent sha3(msg.data, block.number) from referencing malicious files.

## Secure example

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash) {
    require(_data.length > 0);
    // ...}
```

Source lines: 96-414

### ► Finding 4.1: Vulnerability found (High)

#### Vulnerable code

```
`require(url) require('./include.php');` or `require(url + '/index.php');`
```

#### Explanation

Insecure use of `require()` function with unvalidated user input (`url`). This allows remote file inclusion.

#### Impact

Potential code execution and compromise of the system.

#### Entry point

/include.php or /index.php

## Execution path

```
``  
function includeFile($url) {  
    require($url);  
}  
includeFile($_GET['file']);``
```

## Parameters

\$url

## Exploitation steps

- Send a malicious URL as the `file` parameter to `include.php` or `index.php`.

## Example payloads

- `<script>alert('XSS')</script>`

[Return to table of contents](#)

## Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

## Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)